MOBILE APPLICATION DVELOPMENT

Shoaib Farooq

Department of Computer Science

Instructor Information

Instructor	Shoaib Farooq	E-mail	m.shoaib1050@gmail.com
 GitHub	https://github.com/SHOAIB1050		
 YouTube	https://www.youtube.com/c/pakithub		
 LinkedIn	https://www.linkedin.com/in/shoaib-farooq-b5b190105/		

Let's Start .....

Lecture # 6,7


Alibaba.com


ByteDance


ebay


GROUPON


Surface™


kijiji


Google Ads


$d


PHILIPS
hue


Square

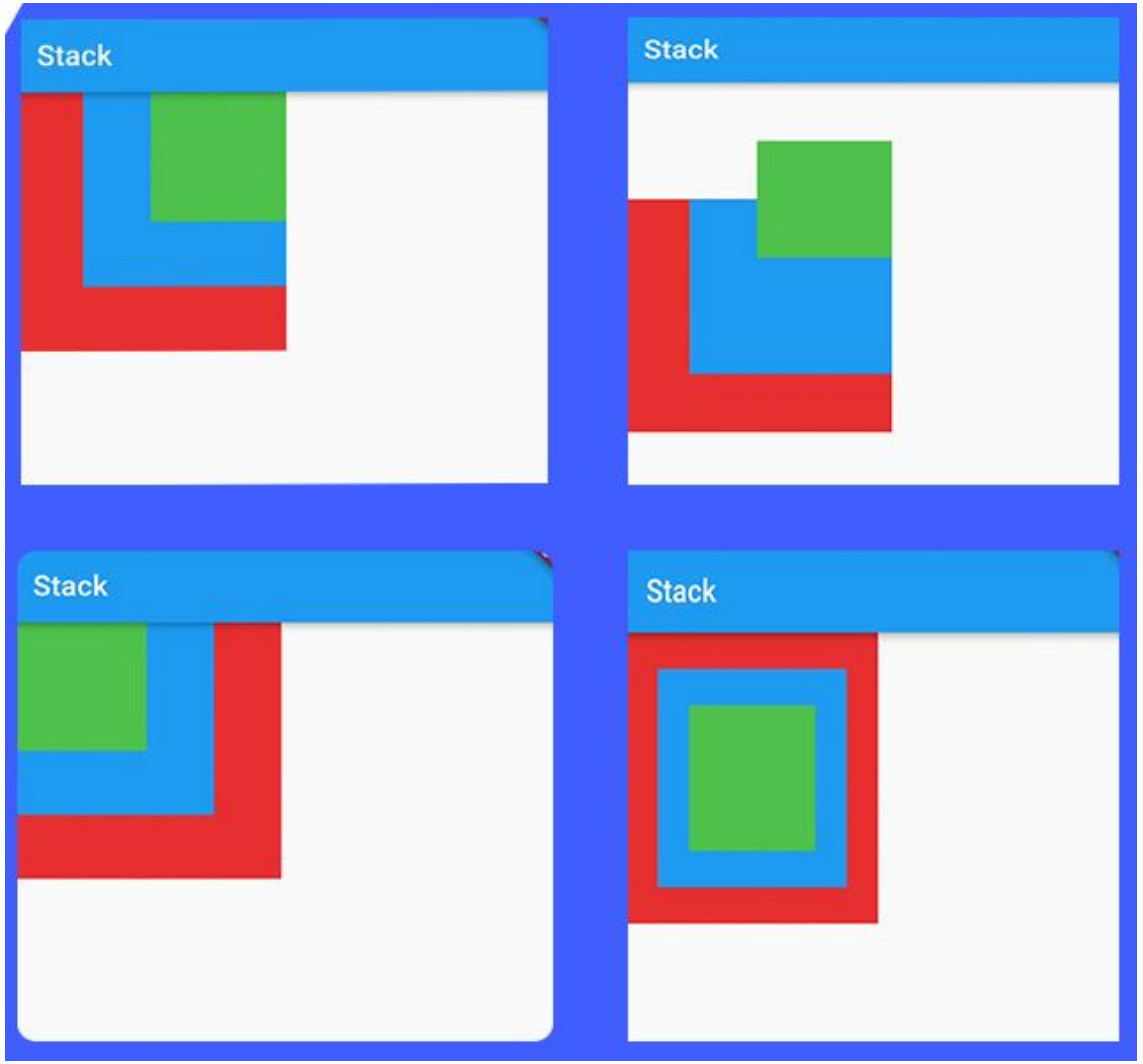
OUTLINE

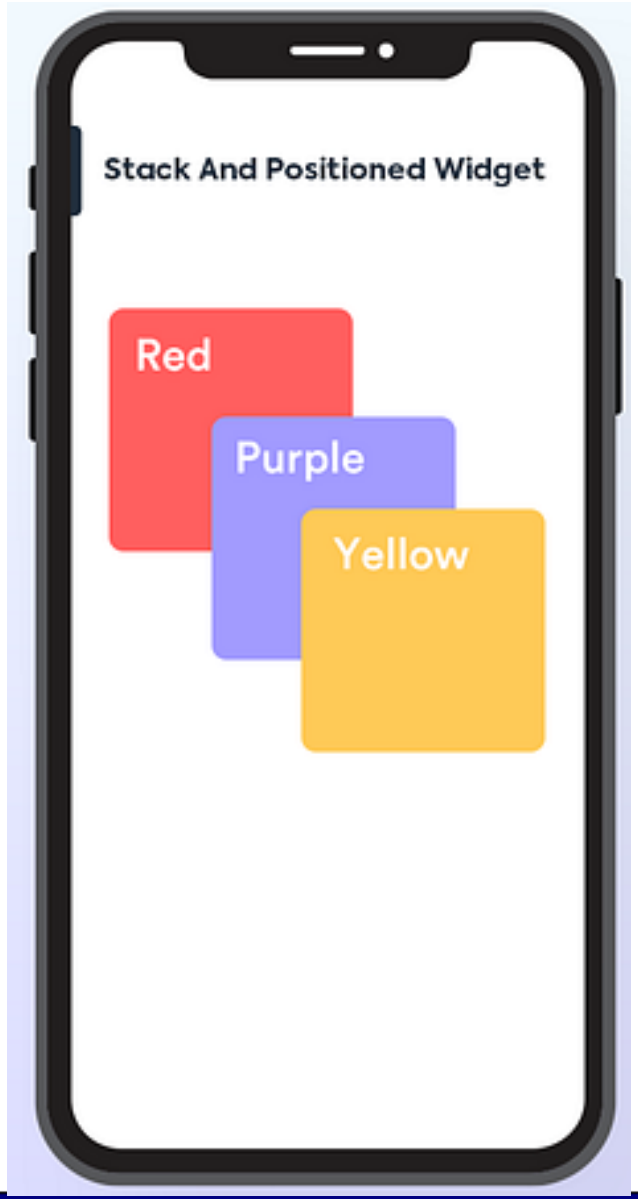
- STACK Widget
- Simple Stack Structure
- Navigation
- Role of Stack in Navigation
- Button Widget
- Types of Buttons in Flutter

STACK WIDGET

- A widget that positions its children relative to the edges of its box.
- This class is useful if you want to overlap several children in a simple way,
- for example having some text and an image, overlaid with a gradient and a button attached to the bottom.

STACK WIDGET





STACK WIDGET

- Stack widget in Flutter overlays multiple widgets on top of each other.
- Allows positioning widgets using absolute coordinates within parent constraints.
- Useful for creating complex layouts with layered elements.
- Ideal for overlapping elements, floating action buttons, or custom designs.

STACK WIDGET

- The **Positioned widget** is used to position its child widget within the Stack. It allows you to specify the top, bottom, left, right, and width/height constraints for positioning the child widget.

- Overall, the Stack widget is powerful for creating complex layouts and overlaying widgets in Flutter.

NAVIGATION

In Flutter, screen navigation can be achieved using the Navigator class and various navigation methods provided by the framework. Here's a basic overview of how screen navigation works in Flutter

PUSHING A NEW SCREEN ONTO THE NAVIGATION STACK

You can navigate to a new screen by pushing a new route onto the navigation stack using the **Navigator.push()** method. For example:

```
Navigator.push(
  context,
  MaterialPageRoute(builder: (context) => SecondScreen()),
);
```

REPLACING THE CURRENT SCREEN

You can replace the current screen with a new one using **the Navigator.pushReplacement()** method. This is useful for scenarios like replacing a login screen with a home screen after successful authentication.

```
Navigator.pushReplacement(
  context,
  MaterialPageRoute(builder: (context) => HomeScreen()),
);
```

POPPING THE CURRENT SCREEN OFF THE NAVIGATION STACK

You can go back to the previous screen by popping the current route off the navigation stack using the **Navigator.pop() method.**

For example, if you have a "Back" button in your UI, you can handle its **onPressed** event like this:

```
onPressed: () {
  Navigator.pop(context);
}
```

PASSING DATA BETWEEN SCREENS:

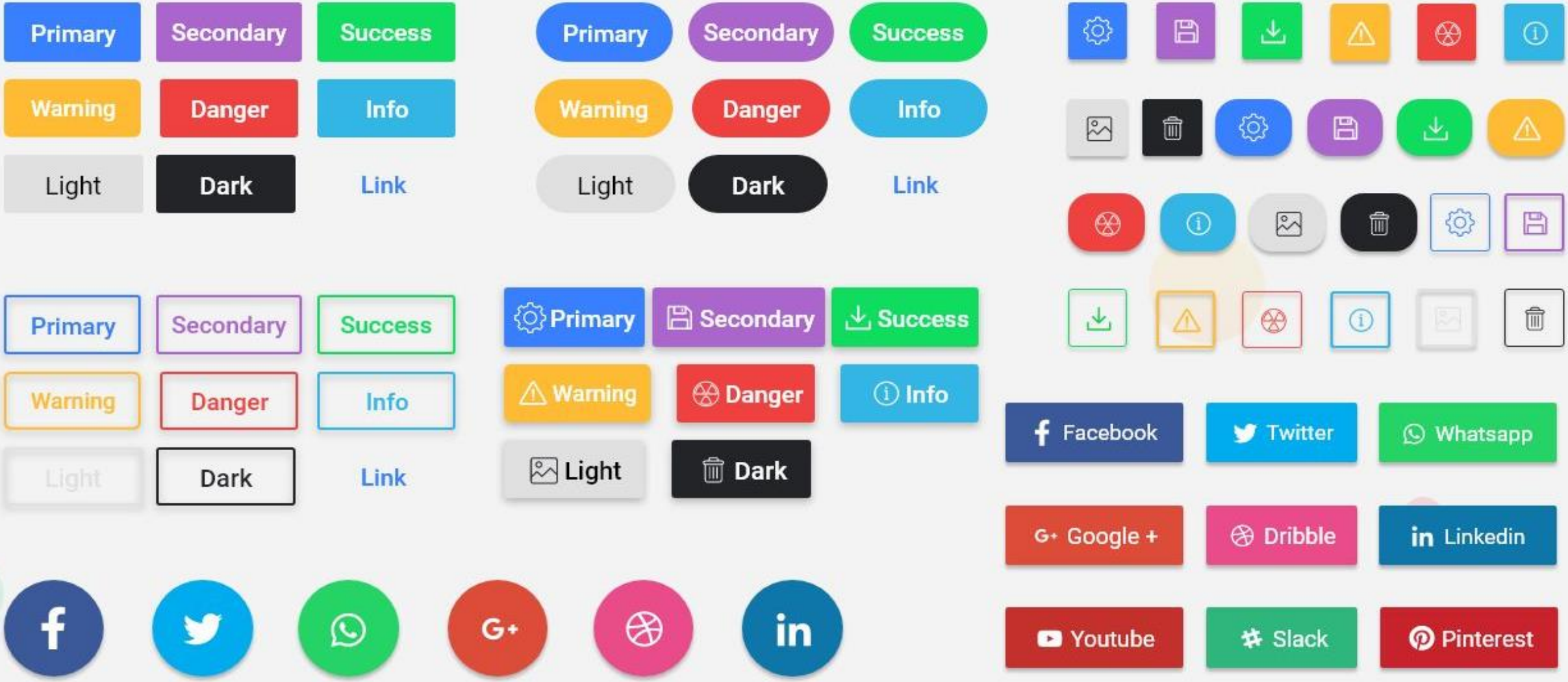
You can pass data between screens by specifying arguments when pushing a new route. For example:

```
Navigator.push(
  context,
  MaterialPageRoute(
    builder: (context) => SecondScreen(data: myData),
  ),
);
```

BUTTON WIDGETS

Flutter provides various types of buttons that you can use to interact with users. Here are some common types of buttons available

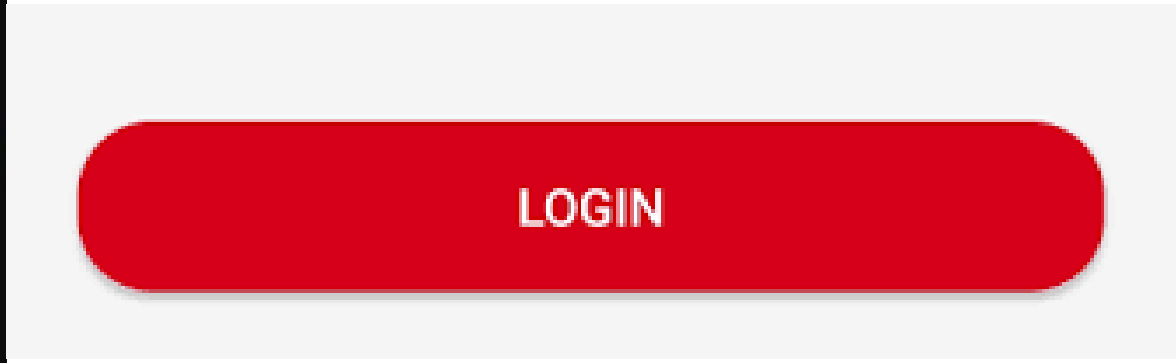
BUTTON WIDGETS



ELEVATED BUTTON

A button with a filled background and elevated appearance when pressed.

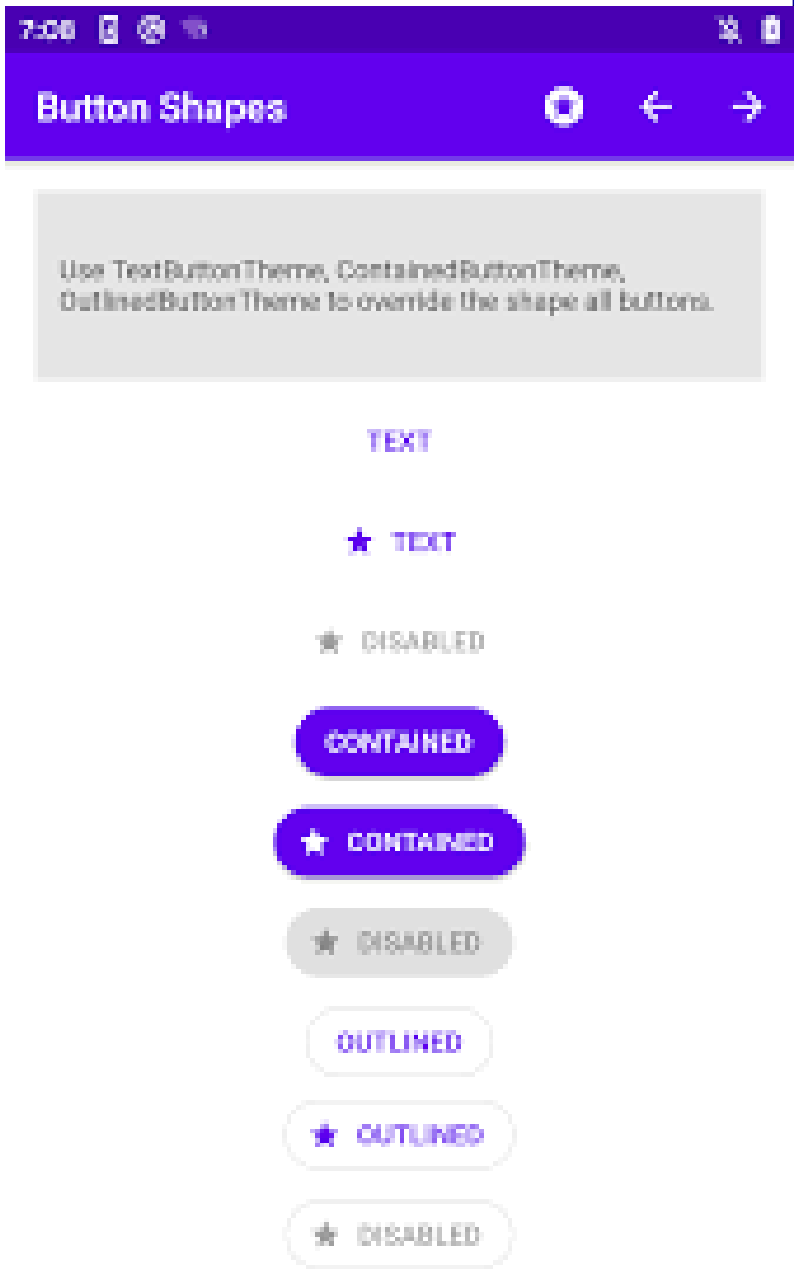
```
ElevatedButton(
  onPressed: () {
    // Action to perform when the button is pressed
  },
  child: Text('Elevated Button'),
)
```



TEXTBUTTON

A button made of text with no background or elevation.

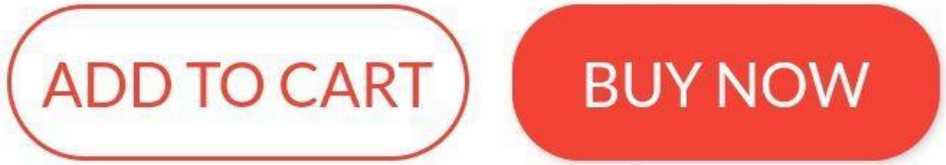
```
TextButton(
  onPressed: () {
    // Action to perform when the button is pressed
  },
  child: Text('Text Button'),
)
```



OUTLINEDBUTTON

In Flutter, an OutlinedButton widget is a button that displays an outlined border around its child, which is typically a text widget. It provides a transparent background, making it suitable for use in scenarios where a button with a visible border is needed.

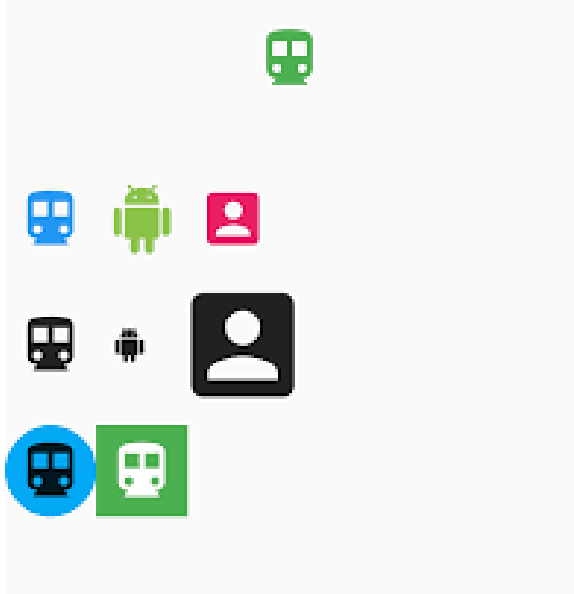
```
OutlinedButton(
  onPressed: () {
    // Action to perform when the button is pressed
  },
  child: Text('Outlined Button'),
)
```



ICONBUTTON

In Flutter, an **IconButton** widget represents a button that typically contains an icon. It's commonly used to provide compact, icon-only actions within the user interface.

```
IconButton(
  onPressed: () {
    // Action to perform when the button is pressed
  },
  icon: Icon(Icons.add),
)
```



FLOATINGACTIONBUTTON

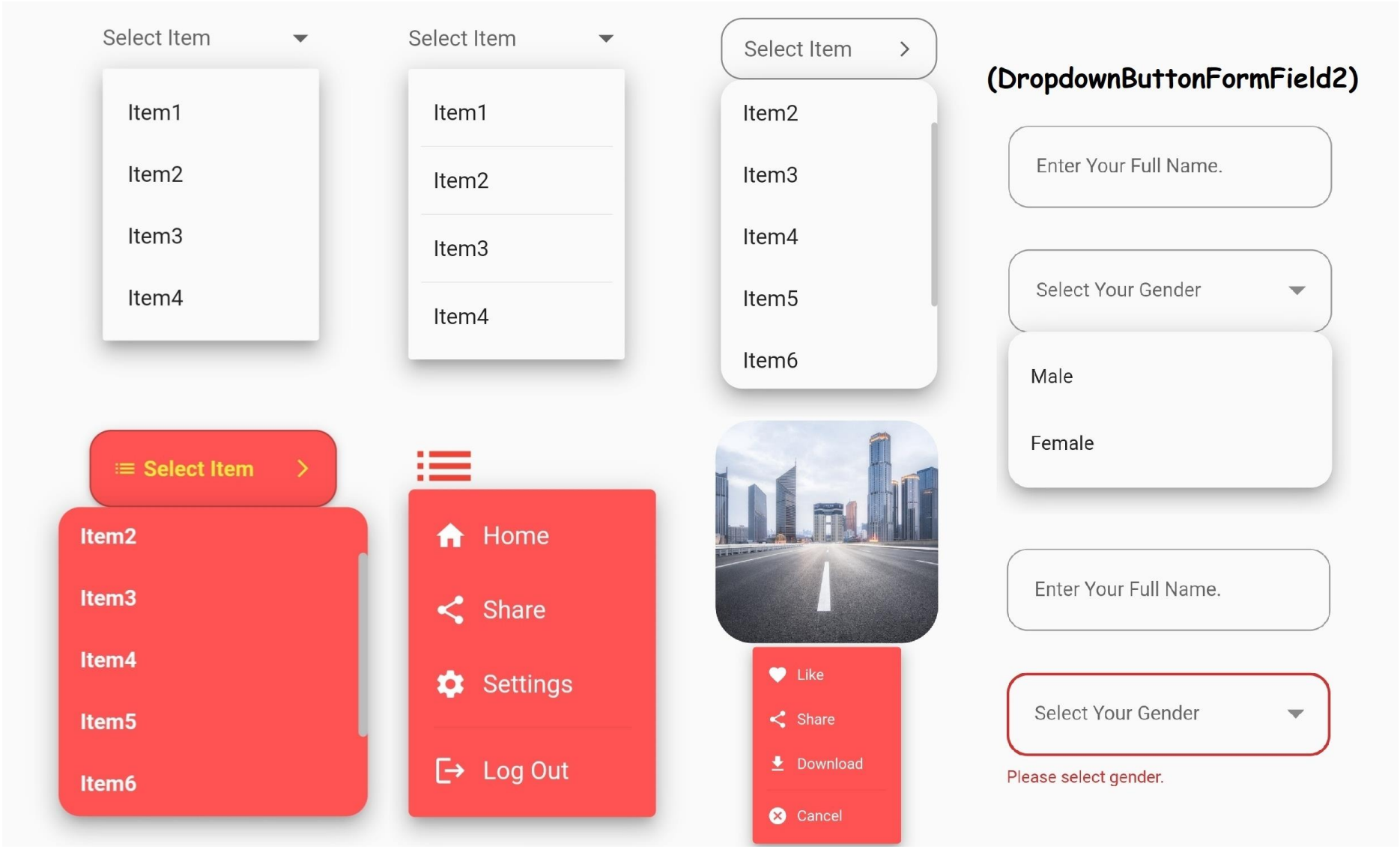
In Flutter, a **FloatingActionButton** widget represents a button that's typically used for a primary action within a scaffold. It's often displayed as a circular button with an icon.

```
FloatingActionButton(
  onPressed: () {
    // Action to perform when the button is pressed
  },
  child: Icon(Icons.add),
)
```

DROPDOWNBUTTON:

In Flutter, a **DropdownButton** widget represents a button that, when pressed, displays a dropdown menu containing a list of items from which the user can select one option. It's commonly used when you want to provide users with a selection of options in a compact and space-efficient manner.

DROPDOWNBUTTON



Refference

https://docs.flutter.dev/ui/layout

Thank You ☺